DRLPlace: A Deep Reinforcement Learning-based Irregular and High-Density Printed Circuit Board Placement Method

Lei Cai[1†], Ke Cheng[1†], Jixin Zhang[1*], Haiyun Li[2], Zhiwei Ye[1]
[1]School of Computer Science, Hubei University of Technology
[2]Tsinghua Shenzhen International Graduate School, Tsinghua University

2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC) | 979-8-3315-9123-6/26/$31.00 ©2026 IEEE | DOI: 10.1109/ASP-DAC66049.2026.11420655

***Abstract*—The placement of components on a printed circuit board (PCB) plays a critical role in modern PCB design workflows. Modern PCBs are characterized by irregular board outlines, significant variations in component sizes, and high component density, which make it challenging for traditional methods to effectively accomplish the placement task. To address these challenges, we propose a deep reinforcement learning-based PCB placement method, capable of handling irregular and high-density PCB with various real-world PCB constraints. Since accurate reward signals are difficult to obtain during the placement process before its completion, which hinders effective optimization guidance, we propose a proximal policy optimization (PPO)-based framework to predict the reward at each step of the placement process. Due to the limited training samples which constrain the reward prediction accuracy in the PPO-process, we extract the layout feature, net feature and density map, and send them into a pre-trained resnet to train a policy-value model to improve the limited accuracy. To improve the efficiency of training convergency, we introduce a chip-centric clustering method for better initialization. We also propose a boundary-aware push-and-reflection-based legalization method to ensure legalization within irregular shaped boundaries. The experimental results show that the proposed method can decrease half-perimeter wire length by up to 16% while handling multiple PCB placement constraints compared to manual designs, significantly exceeding the state-of-the-art PCB placement methods.**

***Index Terms*—Deep Reinforcement Learning, Printed Circuit Board, Placement, Proximal Policy Optimization.**

I. Introduction

Printed circuit board (PCB) placement plays a critical role in modern PCB design workflows and is inherently a challenging nonlinear optimization problem. It involves determining the optimal positions of heterogeneous components, such as chips, resistors, and capacitors, with varying shapes and sizes under intricate constraints, including inter-component spacing and routability. The problem is further complicated by high component density, making it difficult to search for optimal solutions while balancing objectives like wirelength minimization and area utilization.

Traditional PCB placement methods primarily aim to minimize half-perimeter wirelength (HPWL) and placement area using heuristic and stochastic algorithms [1]–[4]. However, due to the challenges in accurately modeling irregularly shaped boundaries, diverse device sizes, and high-density PCB placement regions, these methods often become trapped in local optima and yield suboptimal results that fall short of the requirements of modern PCB designs.

Modern PCB design still heavily relies on iterative manual adjustments by engineers to satisfy these complex constraints. Therefore, we aim to use deep reinforcement learning (DRL) to formulate a modern PCB placement process that emulates the iterative placement process of experienced PCB designers.

However, most contemporary high-efficiency automatic placement approaches including DRL-based methods [5]–[9] and other AI-based methods [10]–[14], are designed for IC placement and are difficult to address the complexities of real-world PCB designs, which involve significantly more intricate constraints. Even recent PCB placement methods [15], [16], which primarily focus on minimizing HPWL, struggle to handle the complexities of modern PCB designs, particularly under irregular boundaries, significant variations in device sizes, and high component density.

The DRL-based PCB placement poses a challenging combinatorial optimization problem characterized by sparse feedback that is only available after full placement, scarce training sample caused by diverse board shapes and device sizes, and unstable reward convergence under complex and constraint-heavy PCB design scenarios.

To address the above three challenges, we propose a novel DRL framework based on proximal policy optimization (PPO), specifically designed to handle the sparse feedback problem. To tackle the issue of scarce training samples, we design a unique PCB feature representation using layout feature, net feature, and density map, which are encoded using a pre-trained ResNet. Additionally, we employ a chip-centric clustering strategy for initial placement generation, which stabilizes reward convergence under complex design scenarios. To ensure physical validity within irregular board outlines, we further develop a boundary-aware push-and-reflection legalization mechanism.

Our contributions are as follows:

- We propose DRLPlace, explicitly modeling devices with various sizes while incorporating multiple constraints such as irregular boundary and high-density space during the placement process.

† Equal Contribution.
* Corresponding authors: Jixin Zhang {zhangjx@hbut.edu.cn}

- We propose a PCB feature representation and a PCB feature encoder using pre-trained ResNet to guide reward prediction and policy learning in sparse-reward settings.
- We design a chip-centric clustering method for optimal initialization, and a boundary-aware push-and-reflection legalization mechanism for handling irregular boundaries.
- We conduct experiments on real-world commercial PCB designs, and the results show that our DRLPlace can achieve better HPWL on average while handling multiple PCB constraints compared to manual placement, significantly exceeding state-of-the-art (SOTA) method.

II. Related Works

A. Traditional PCB Placement Methods

Traditional PCB placement has been widely studied using heuristic algorithms such as genetic algorithms (GAs), particle swarm optimization (PSO), and simulated annealing (SA). Jain et al. [1] applied GAs to minimize wire length, while Badriyah et al. [3] combined GAs with Lee's routing to jointly optimize placement and routing. To address multiple objectives such as thermal constraints and area usage, Ismail et al. [2] proposed a self-organizing GA with hierarchical fitness evaluation. Alexandridis et al. [4] used a thermal-aware PSO to reduce temperature hotspots. Recently, SA-PCB [17] employed simulated annealing in an open-source framework for general PCB layout tasks. However, these methods require significant manual tuning and lack adaptability to modern PCB design constraints, motivating the development of more efficient and flexible alternatives.

B. Modern PCB Placement Methods

To date, most modern placement methods have focused on Integrated Circuit (IC) placement, with limited exploration in the PCB domain. Vassallo et al. [15] proposed a reinforcement learning framework with adaptive rewards for PCB placement, demonstrating that adaptive reward shaping can guide the agent toward efficient placement policies. However, it still focuses on small-scale PCB designs. Cheng et al. [16] proposed a hybrid framework that formulates PCB placement as a max-margin optimization problem to enhance net separation. Their method combines seed layout generation, coordinate-descent refinement, and a mixed-integer linear programming legalization stage. Nonetheless, these PCB placement approaches have not effectively adapted to modern PCB designs, which are characterized by irregularly shaped boundaries, highly diverse device sizes, and extremely high-density placement regions. As a result, they continue to face significant challenges when applied to real-world, constraint-intensive PCB scenarios.

III. Problem Formulation

The PCB placement problem can be formulated as follows: Given an irregular PCB boundary $\mathcal{B} \subset \mathbb{R}^2$, a set of PCB design constraints $\mathcal{C}$, a set of devices $\mathcal{D}$ with diverse sizes, a set of pins $\mathcal{P}$, a set of nets $\mathcal{N}$, and a set of module groups $\mathcal{MG}$, determine the coordinates and orientations of all devices considering the PCB constraints $\mathcal{C}$ such that all devices are within the irregular boundary $\mathcal{B}$, no devices overlap, and all design requirements are met. The details of all definitions are shown in Table I.

TABLE I
Notation and Key Terms

Symbol	Description
$\mathcal{D} = \{D_1, D_2, \ldots, D_n\}$	Set of devices
$\mathcal{P} = \{p_1, p_2, \ldots, p_m\}$	Set of pins
$\mathcal{N} = \{N_1, N_2, \ldots, N_k\}$	Set of nets
$\mathcal{MG} = \{G_1, G_2, \ldots, G_l\}$	Set of module groups
$\mathcal{B} \subset \mathbb{R}^2$	Irregular PCB boundary
$\mathcal{C}$	Set of PCB constraints
$\mathcal{L} = \{(x_i, y_i, \theta_i)\}_{i=1}^{n}$	Final placement solution

A. Objective of PCB Placement

The objective of PCB placement is to determine optimal coordinates and orientations for all devices within a highly irregular boundary while satisfying practical design constraints. Given devices with significant size variation, the placement must minimize wire length, while promoting spatial compactness and routability.

Formally, we aim to minimize a composite cost function:

$$\min \quad \mathcal{L} = \text{Density} + \text{Wirelength}, \tag{1}$$

subject to that constraints $\mathcal{C}(D, \mathcal{P}, \mathcal{N}, \mathcal{MG})$ must be satisfied.

B. Constraints of PCB Placement

- **Irregular Boundary:** All devices must be placed entirely within the irregular PCB boundary $\mathcal{B}$:

$$\forall D_i \in \mathcal{D}, \quad D_i \subseteq \mathcal{B}. \tag{2}$$

- **Spacing:** A minimum spacing $d_{\min}$ must be maintained between any two devices:

$$\text{dist}(D_i, D_j) \geq d_{\min}, \quad \forall D_i \neq D_j. \tag{3}$$

- **Overlap:** Device footprints must not overlap, even under extreme density:

$$D_i \cap D_j = \emptyset, \quad \forall D_i \neq D_j. \tag{4}$$

- **Design Rule Compliance:** All placements must conform to DRC rules; violations result in invalid layouts.

IV. Methodology

A. Framework of DRLPlace

To tackle irregular shapes, high component density, and design constraints in modern PCB layout, we propose a deep reinforcement learning framework, shown in Fig. 1, using a PPO backbone. We improve learning by extracting and encoding layout features, net features, and density maps with a pre-trained ResNet. This encoding trains a shared policy-value model. Additionally, our chip-centric clustering strategy enhances initial placements and speeds up convergence. A push-and-reflection legalization algorithm ensures boundary-aware placement validity.

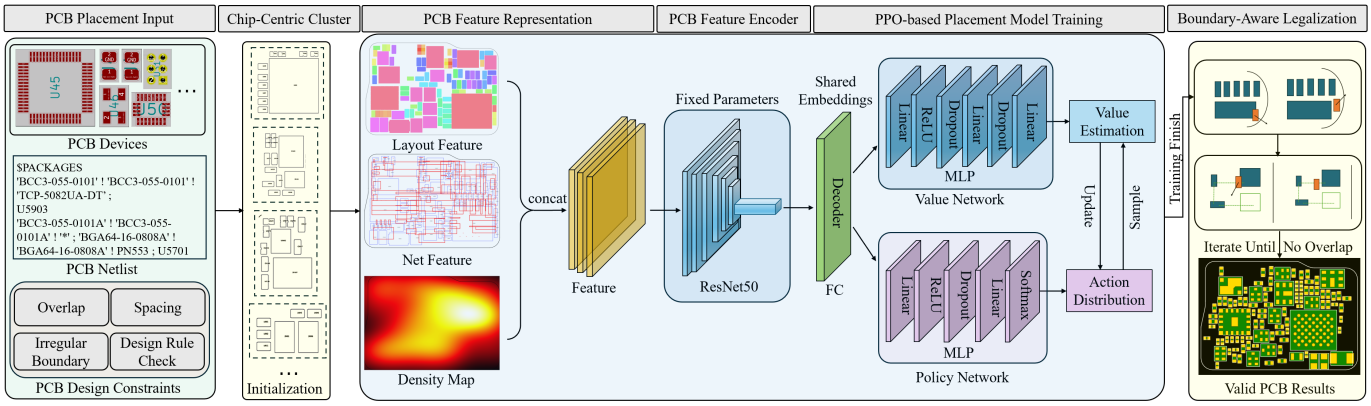


Fig. 1. Framework of DRLPlace.

B. Chip-Centric Cluster

At the beginning of placement, all devices are initialized within the irregular boundary. To promote a better initial layout state of the placement process, we apply a chip-centric clustering algorithm (see Algorithm 1) to group functionally related components around major chips.

Algorithm 1 Chip-Centric Clustering Algorithm

Require: Netlist information $\mathcal{N}$, device list $\mathcal{R}$, target cluster size $\mathcal{L}$
Ensure: Device cluster for module group $\mathcal{MG}$

```
Initialize cluster set MG ← ∅
while there are unclustered devices in R do
    Initialize a new cluster s with the largest unclustered device r_i in R
    while size of s is less than L do
        Find the smallest unclustered device r_j in R connected to r_i based on netlist N
        Add r_j to cluster s
        if all devices have been clustered then
            break
        end if
    end while
    Add cluster s to MG
end while
return MG
```

C. PCB Feature Representation

To capture PCB layout characteristics, we adopt a multi-modal representation comprising a layout feature, net feature and density map. As shown in Fig. 1, the layout feature distinguishes components by function (e.g., capacitor, resistor, IC), aiding placement decisions. The net feature encodes net connectivity to reveal routing patterns. The density map indicates layout congestion by showing occupied regions.

D. PCB Feature Encoder

To effectively encode the PCB layout state, we construct the PCB feature input as a combination of three components: the layout feature $\mathbf{I}_{\text{layout}}$, the net feature $\mathbf{I}_{\text{net}}$, and the density map $\mathbf{I}_{\text{density}}$. These features are concatenated along the channel to form a unified tensor:

$$\mathbf{X}_{\text{input}} = \text{Concat}\left(\mathbf{I}_{\text{layout}},\ \mathbf{I}_{\text{net}},\ \mathbf{I}_{\text{density}}\right). \tag{5}$$

The input $\mathbf{X}_{\text{input}}$ is then processed by a pretrained ResNet50 [18] backbone to extract high-level spatial features:

$$\mathbf{F}_{\text{resnet}} = \text{ResNet}(\mathbf{X}_{\text{input}}). \tag{6}$$

Finally, the extracted features are passed through a fully connected layer FC to generate the input feature embeddings for the policy and value networks:

$$\mathbf{E}_{\text{RL}} = \text{FC}(\mathbf{F}_{\text{resnet}}) \tag{7}$$

To reduce training overhead and fully exploit the feature extraction capability of the pretrained ResNet50, we freeze its weights and exclude it from gradient updates. Only the parameters of the fully connected layers and the downstream policy and value networks are trained.

E. PPO-based Placement Model Training

1) Constrained Action Space: To ensure valid and manufacturable placements, we design a constrained action space. Each device's actions, including translation and rotation, are restricted to prevent crossing the PCB boundary:

$$\mathcal{A}_{\text{valid}} = \{a \in \mathcal{A} \mid a(D_i) \subseteq \mathcal{B},\ \forall i\}. \tag{8}$$

Only orthogonal rotations (e.g., $0°, 90°, 180°, 270°$) are allowed to reduce routing complexity and preserve design rules.

2) Multi-Objective Reward Formulation: The overall reward function r_t is designed to balance key placement objectives: overlap, wire length, and density. It is defined as:

$$r_t = \alpha \cdot r_t^{\text{overlap}} + \beta \cdot r_t^{\text{wirelength}} + \gamma \cdot r_t^{\text{density}}, \tag{9}$$

where α, β and γ are weighting coefficients that balance the contribution of each objective.

a) Overlap Reward: The overlap reward r_t^{overlap} is computed by the total overlapping area among all placed devices at time step t. Specifically, we penalize the devices for any overlap to encourage legal placement. The reward is defined as a negative function of the overlap area:

$$r_t^{\text{overlap}} = -\sum_{i \neq j}^{n} (D_i \cap D_j), \tag{10}$$

where $D_i \cap D_j$ is the intersection area between D_i and D_j.

b) Density Reward: To estimate layout density under irregular PCB boundaries and significant variations in device sizes, we adopt a density reward. The local density at time step t is defined as:

$$r_t^{\text{density}} = -\sum_{i \neq j}^{n} \max\left(r_{ij} - \text{dist}(D_i, D_j), 0\right), \tag{11}$$

where $\text{dist}(D_i, D_j)$ is the distance between each other, and r_{ij} represents the maximum separation threshold. The density reward applies a penalty to discourage configurations for one device is entirely contained within another.

c) Wirelength Reward: The wire length reward $r_t^{\text{wirelength}}$ is computed using the Half-Perimeter Wire Length (HPWL) of each net:

$$r_t^{\text{wirelength}} = -\sum_{N_i \in \mathcal{N}} \text{HPWL}(N_i), \tag{12}$$

$$\text{HPWL}(N_i) = \max_{p_k, p_j \in N_i} \left(|x_k - x_j|\right) + \max_{p_k, p_j \in N_i} \left(|y_k - y_j|\right), \tag{13}$$

where $\mathcal{N}$ denotes the set of all nets.

3) Self-Supervised Value Estimation: To estimate long-term returns in the presence of sparse and delayed feedback, we adopt a self-supervised value network $V_\phi(s)$ trained from historical trajectories. Given a trajectory $\tau = \{(s_t, a_t, r_t)\}_{t=1}^{T}$, the target return $\hat{R}_t$ is computed for each state s_t, and the value network is optimized using the mean squared error loss:

$$\mathcal{L}_V(\phi) = \frac{1}{T} \sum_{t=1}^{T} \left(V_\phi(s_t) - \hat{R}_t\right)^2, \tag{14}$$

where Generalized Advantage Estimation is used for computing $\hat{R}_t$, and the bootstrapped return $\hat{R}_t$ is computed as:

$$\hat{R}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \text{where } \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \tag{15}$$

and this allows the value function to be trained with dense feedback signals derived from sparse rewards.

4) Proximal Policy Optimization with Sparse Feedback: The policy network $\pi_\theta(a_t \mid s_t)$ is trained using the PPO algorithm [19]. The environment state s_t is defined by the current placement configuration. The action a_t represents a discrete translation and rotation applied to a selected component. And the policy is updated by maximizing the clipped surrogate objective:

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \hat{A}_t,\ r\hat{A}_t\right)\right], \tag{16}$$

$$r = \text{clip}\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon\right), \tag{17}$$

where $\hat{A}_t = \hat{R}_t - V_\phi(s_t)$, and $V_\phi(s_t)$ is the output of value network. This iterative training process refines both the policy π_θ and the value function V_ϕ based on real environment feedback and self-supervised signals, leading to progressively improved placement performance.

Algorithm 2 Push-and-reflection legalization process

Require: Device list $\mathcal{D}$, boundary $\mathcal{B}$,
Ensure: Updated placement with reduced overlaps

while $\exists\ D_i, D_j \in \mathcal{D}$ and $D_i \cap D_j > 0$ **do**
  Compute $v_{ij} = p_i - p_j$.
  Normalize: $u_{ij} = v_{ij}/(\parallel v_{ij} \parallel + \epsilon)$
  With probability 0.1 for random perturbation, scale u_{ij}.
  Form $\Delta_i = u_{ij} - (w_{dx}[i], w_{dy}[i])$.
  Attempt direct move: $p_i' = p_i + \Delta_i$
  while p_i' is invalid **do**
    find boundary edge intersecting the ray (p_i, d),
    reflect d as above,
    set $\widetilde{\Delta_i} = \frac{d}{\|d\|+\epsilon}\rho - (w_{dx}[i], w_{dy}[i]), \rho \sim (0,1)$
    Attempt move: $p_i' = p_i + \Delta_i$
  end while
  set $d_i \longleftarrow p_i'$
end while

F. Boundary-Aware Legalization

The legalization process refines the placement result by resolving overlaps and enforcing irregular boundary constraints. We propose a novel push-and-reflection-based legalization method, as detailed in Algorithm 2. For each pair of overlapping devices, a repulsive displacement vector is computed based on their relative positions and gradient history. A small probability of random perturbation is added to avoid local minima. If the proposed move leads to an invalid position, we perform a reflection along the nearest boundary edge to ensure the displacement remains within the legal region. This method efficiently reduces overlaps while preserving the global placement structure and minimizing the impact on HPWL, considering the irregular PCB boundary and constraints.

V. Experiment

A. Experiment Setup

We conducted experiments on a Linux machine with an Intel Core i9-14900K and an Nvidia RTX 4090. The learning rates of the value network and the policy network are both set to 1e-5. The rates α, β, and γ for balancing the contribution of each objective are set to 0.5, 1, and 1, respectively.

B. Experiment Dataset

We conducted experiments on 10 commercial PCB designs originating from Huawei Terminal Co., Ltd, all of which are compared with manual placement results performed by

expert PCB designers. Table III summarizes the characteristics of the commercial PCB placement benchmarks used in our experiments. Each case contains varying numbers of devices (ranging from 33 to 277) and chips (from 2 to 4), simulating different levels of integration complexity. The Space(%) column indicates the ratio of device area to total layout area, serving as a proxy for placement density. Most cases have a density above 80%, posing a significant challenge for overlap-free and wirelength-efficient placement. The benchmark set includes both small-scale (e.g., case2) and large-scale (e.g., case9) layouts, thus offering a comprehensive evaluation for the scalability and generalizability of the proposed algorithm.

Besides, we also conducted DRLPlace on open-source PCB datasets [15], which are adapted to the SOTA DRL method [15]. We evaluate the wire length of the results using the open-source router [20].

TABLE II
DESIGN CHARACTERISTICS

Case	Devices	Chips	Space(%)	Area(mm^2)	Nets	Pins
case1	33	2	79.5	99998	30	136
case2	41	3	82.4	28624	32	100
case3	35	2	81.2	78809	26	126
case4	35	2	80.5	38545	30	100
case5	97	4	82.6	205001	138	666
case6	57	3	81.3	59146	34	152
case7	63	3	85.1	106836	53	185
case8	77	3	80.2	179521	69	231
case9	261	4	87.5	991850	439	1737
case10	277	4	87.1	618503	185	1117

C. Methods for Comparison

We compare DRLPlace with SOTA methods, including reinforcement learning PCB placement [15] and simulated annealing PCB placement [17]. For a fair comparison, the number of their iterations is set to 300k. The manual results are considered as baselines for these methods. The evaluated industrial cases were designed by experienced engineers, and the manual runtimes, estimated by experts, represent lower bounds under optimal conditions. In practice, actual efforts are typically longer due to iterative adjustments and verification.

D. Result Analysis For Commercial PCB Datasets

Table III presents a comparison of HPWL, runtime (T), and overlap ratio (i.e., the proportion of overlapping area relative to the total area) across SOTA PCB placement methods, including manual placement, SA-PCB [17], the RL-based approach [15], and DRLPlace.

As illustrated in Table III, SA-PCB and RL often lead to significantly higher HPWL, averaging 53.92% and 38.12% more than manual designs, respectively. These methods lack fine-grained placement heuristics and struggle to adapt to the complexities of modern PCB layouts, resulting in suboptimal routing performance. In contrast, our proposed method achieves an average HPWL improvement of -0.04% compared to manual placement. This remarkable performance is attributed to the high-precision policy network, which learns to effectively capture spatial correlations and optimize placement decisions. By leveraging PCB feature representation and end-to-end training, our approach consistently enhances layout compactness, even under irregular boundaries and dense component distributions.

Regarding the overlap ratio, both the manual placement and DRLPlace consistently achieve **0%** overlap across all test cases, indicating fully legal placements without any component collisions. In contrast, the SA-PCB and RL-based approaches yield average overlap ratios of **5.48%** and **5.03%**, respectively. Particularly in complex designs such as `case5`, `case6`, and `case10`, these methods produce excessive overlaps exceeding **8%–11%**, demonstrating their inability to handle high-density and constraint-heavy layouts. These results underscore the strength of DRLPlace's PPO-based framework in effectively satisfying physical design constraints.

In terms of runtime, DRLPlace is also highly competitive. Compared to SA-PCB and RL, it achieves average runtime reductions of 30% and 16%, respectively. While the runtime slightly exceeds manual layout in some simpler cases, it demonstrates substantial time savings in more complex designs (e.g., `case2`, `case3`). This improvement stems from our chip-centric initialization strategy, which accelerates convergence by providing a reasonable starting point for optimization. Moreover, the entire placement process is fully automated, eliminating the need for manual intervention and making it well-suited for scalable industrial deployment.

In summary, DRLPlace outperforms SOTA and manual designs across complex scenarios, proving effective for real-world PCBs with irregular outlines, diverse component sizes, and high density. Unlike prior DRL approaches [15], DRLPlace overcomes sparse feedback, handles design variation, and ensures stable convergence under complex constraints.

E. Result Analysis For Open-Source PCB Datasets

Table IV presents a quantitative comparison of different methods on nine open-source PCB benchmarks. We evaluate the original PCB designs, SA-PCB [17], RL [15], and DRLPlace in terms of HPWL, actual routing wire length (WL), runtime (T), and overlap ratio.

DRLPlace consistently achieves competitive performance in both HPWL and WL compared to SOTA PCB placement methods. On average, DRLPlace reduces WL and HPWL more effectively than both RL and SA-PCB. Notably, in challenging cases such as `pcb3`, `pcb4`, and `pcb6`, the WL reduction exceeds **60%**, accompanied by significant HPWL improvements. These results underscore the high precision and accuracy of DRLPlace's placement model, which directly contribute to superior WL and HPWL outcomes across a wide range of open-source PCB layouts.

Despite significant improvements in both WL and HPWL, DRLPlace maintains a competitive average runtime of **29.65 minutes**, achieving a **4× speedup** over RL (117.60 minutes) and slightly outperforming SA-PCB (32.31 minutes) in efficiency, while delivering substantially better placement quality.

TABLE III
EXPERIMENTAL RESULTS COMPARISON ON COMMERCIAL DATASETS

Case	Manual			SA-PCB [17]			RL [15]			DRLPlace (ours)		
	HPWL(mm)	Overlap(%)	T(h)	HPWL(mm)	Overlap(%)	T(h)	HPWL(mm)	Overlap(%)	T(h)	HPWL(mm)	Overlap(%)	T(h)
case1	2222.63	0	≥1.50	3284.39	1.6	2.71	2956.87	0.7	1.92	**1850.86**	**0**	**0.75**
case2	858.88	0	≥1.00	1758.96	1.4	2.10	1659.27	0.7	1.70	**865.02**	0	**0.58**
case3	2092.49	0	≥1.50	3284.39	2.5	2.71	2956.87	1.1	1.92	**2105.74**	**0**	**1.34**
case4	804.26	0	≥1.00	1784.91	4.1	2.26	1489.72	2.3	1.76	**832.69**	**0**	**0.62**
case5	7273.15	0	≥**6.00**	9952.84	11.2	15.60	8259.14	8.7	12.20	**7690.73**	**0**	7.82
case6	5274.16	0	≥4.00	8822.17	3.1	7.10	7168.48	5.4	5.60	**5175.89**	**0**	**5.13**
case7	1811.88	0	≥3.00	2527.98	5.3	5.40	2708.17	4.7	4.10	**1896.63**	**0**	**2.67**
case8	5695.70	0	≥7.50	6894.96	6.9	12.10	6259.53	6.8	9.70	**5860.51**	**0**	**6.10**
case9	70800.72	0	≥**20.00**	84726.91	10.4	36.80	77840.69	8.3	29.40	**67782.79**	**0**	28.80
case10	36406.63	0	≥**15.00**	44894.06	8.3	25.60	39973.63	11.6	23.20	**37893.05**	**0**	23.40
Avg.	13324.05	0	≥6.05	16793.16	5.48	11.238	15127.23	5.03	9.14	**13195.39**	**0**	**7.72**

TABLE IV
EXPERIMENTAL RESULTS COMPARISON ON OPEN-SOURCE DATASETS [15]

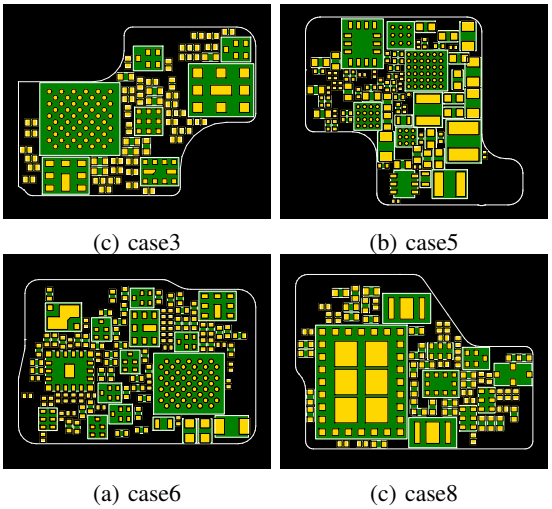
Case	Original PCB		SA-PCB [17]				RL [15]				DRLPlace (ours)			
	WL(mm)	Overlap(%)	HPWL(mm)	WL(mm)	Overlap(%)	T(min)	HPWL(mm)	WL(mm)	Overlap(%)	T(min)	HPWL(mm)	WL(mm)	Overlap(%)	T(min)
pcb1	37.95	0	28.53	38.52	1.6	33.71	26.87	33.01	0	126.92	**27.70**	**37.81**	0	**26.75**
pcb2	44.46	0	16.44	28.93	0	37.10	19.27	**23.41**	0.8	116.70	**10.68**	28.18	0	**34.58**
pcb3	53.25	0	15.52	25.28	2.3	30.71	16.87	28.84	1.5	121.92	**10.86**	**20.69**	**0**	**31.34**
pcb4	70.74	0	25.71	43.08	2.1	35.26	29.72	45.61	0	133.76	**20.14**	**41.84**	0	**15.62**
pcb5	37.05	0	13.59	21.20	4.2	34.60	**12.14**	20.55	1.8	112.20	15.72	**17.84**	**0**	**20.82**
pcb6	76.77	0	22.26	35.60	3.5	32.10	25.48	30.92	2.1	125.60	**18.25**	**30.14**	**0**	**35.13**
pcb7	54.59	0	30.92	38.47	2.7	34.40	33.17	**29.43**	2.2	114.10	**28.1**	36.57	**0**	**32.67**
pcb8	29.70	0	**10.83**	**13.47**	0.6	31.10	19.53	20.83	3.2	99.70	13.67	17.21	**0**	**36.10**
pcb9	91.30	0	48.58	75.30	0.3	30.80	47.69	55.96	4.1	109.40	**42.65**	**55.08**	**0**	**28.80**
Avg.	55.09	0	23.60	35.54	1.922	32.31	25.64	32.06	1.744	117.60	**20.31**	**31.71**	**0**	**29.65**

The relatively slow performance of RL [15] stems from its off-policy learning paradigm, which relies heavily on offline data collection and experience replay, leading to inefficient policy updates. In contrast, DRLPlace benefits from a chip-centric initialization strategy that accelerates convergence by providing better initial placement priors.

DRLPlace consistently achieves zero overlap ratio across all benchmark cases, while the RL-based method [15] and SA-PCB [17] produce non-negligible overlaps in several cases due to they are highly sensitive to hyperparameters and lack of legalization strategies. Overall, DRLPlace achieves the best balance among WL, HPWL, overlap ratio, and runtime across all benchmarks. Its consistent performance demonstrates strong generalizability and robustness, with competitive results on open-source datasets, making it practical for both commercial and academic applications.

F. Case Study

Fig. 2 shows placement results of DRLPlace on real-world PCB benchmarks from Huawei Terminal Co., Ltd. Modern PCBs feature irregular outlines, high component density, and diverse device sizes, making placement particularly challenging due to limited usable space. Despite these challenges, DRLPlace consistently produces compact, non-overlapping layouts that fully satisfy all design constraints. Compared to manual placement, DRLPlace improved wire length metrics and achieves better efficiency. These results demonstrate the practical effectiveness of DRLPlace in handling dense, irregular, and constraint-rich modern PCB designs.



(c) case3 (b) case5

(a) case6 (c) case8

Fig. 2. Visualization of the real-world PCB results created by DRLPlace.

VI. CONCLUSION

In this work, we present a novel deep reinforcement learning-based PCB placement method that effectively addresses the challenges of modern PCB design, including irregular boundary, diverse component sizes, and high-density constraints. Leveraging a PPO framework with stepwise reward prediction, ResNet-extracted layout features, chip-centric initialization, and boundary-aware legalization, our method outperforms both manual and SOTA approaches. In the future, we will extend our work to more complex and larger scales of modern PCB designs.

ACKNOWLEDGMENT

This work is partially supported by the National Natural Science foundation of China under Grant No. 62441233, 62002106, and Huawei Terminal Co., Ltd.

REFERENCES

[1] S. Jain and H. C. Gea, "Pcb layout design using a genetic algorithm," *Journal of Electronic Packaging*, vol. 118, pp. 11–15, 1995. [Online]. Available: https://api.semanticscholar.org/CorpusID:110688179

[2] F. S. Ismail, R. Yusof, and M. Khalid, "Optimization of electronics component placement design on pcb using self organizing genetic algorithm (soga)," *J. Intell. Manuf.*, vol. 23, no. 3, p. 883–895, Jun. 2012. [Online]. Available: https://doi.org/10.1007/s10845-010-0444-x

[3] T. Badriyah, F. Setyorini, and N. Yuliawan, "The implementation of genetic algorithm and routing lee for pcb design optimization," in *2016 International Conference on Informatics and Computing (ICIC)*, 2016, pp. 148–153.

[4] A. Alexandridis, E. Paizis, E. Chondrodima, and M. Stogiannos, "A particle swarm optimization approach in printed circuit board thermal design," *Integr. Comput.-Aided Eng.*, vol. 24, no. 2, p. 143–155, Jan. 2017. [Online]. Available: https://doi.org/10.3233/ICA-160536

[5] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nazi, J. Pak, A. Tong, K. Srinivasa, W. Hang, E. Tuncer, Q. V. Le, J. Laudon, R. Ho, R. Carpenter, and J. Dean, "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, 2021.

[6] R. Cheng and J. Yan, "On joint learning for solving placement and routing in chip design," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 16 508–16 519.

[7] Y. Lai, Y. Mu, and P. Luo, "Maskplace: Fast chip placement via reinforced visual representation learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 019–24 030, 2022.

[8] Y. Lai, J. Liu, Z. Tang, B. Wang, J. Hao, and P. Luo, "Chipformer: Transferable chip placement via offline decision transformer," in *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 18 346–18 364.

[9] K. Xue, R.-T. Chen, X. Lin, Y. Shi, S. Kai, S. Xu, and C. Qian, "Reinforcement learning policy as macro regulator rather than macro placer," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 140 565–140 588.

[10] A. Gusmo, R. Póvoa, N. Horta, N. Loureno, and R. Martins, "Deepplacer: A custom integrated opamp placement tool using deep models," *Appl. Soft Comput.*, vol. 115, p. 108188, 2021.

[11] J. Lu, L. Lei, J. Huang, F. Yang, L. Shang, and X. Zeng, "Automatic op-amp generation from specification to layout," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 12, pp. 4378–4390, 2023.

[12] K. Zhu, H. Chen, W. J. Turner, G. F. Kokai, P.-H. Wei, D. Z. Pan, and H. Ren, "Tag: Learning circuit spatial embedding from layouts," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD '22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: https://doi.org/10.1145/3508352.3549384

[13] Z. Shi, H. Pan, S. Khan, M. Li, Y. Liu, J. Huang, H.-L. Zhen, M. jie Yuan, Z. Chu, and Q. Xu, "Deepgate2: Functionality-aware circuit representation learning," *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pp. 1–9, 2023.

[14] Y. Li, Y. Lin, M. Madhusudan, A. Sharma, W. Xu, S. S. Sapatnekar, R. Harjani, and J. Hu, "A customized graph neural network model for guiding analog ic placement," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–9.

[15] L. Vassallo and J. Bajada, "Learning circuit placement techniques through reinforcement learning with adaptive rewards," in *2024 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2024, pp. 1–6.

[16] C.-K. Cheng, C.-T. Ho, and C. Holtz, "Net separation-oriented printed circuit board placement via margin maximization," in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2022, pp. 288–293.

[17] The OpenROAD Project. (2020) SA-PCB: Simulated Annealing-based Placement For PCB Layout. [Online; accessed 9-June-2025]. [Online]. Available: https://github.com/The-OpenROAD-Project/SA-PCB

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[20] T.-C. Lin, C. Holtz, Yenyi, and D. J. Merrill, "The OpenROAD project - printed circuit board (PCB) router," GitHub, 2020, [Online]. [Online]. Available: https://github.com/The-OpenROAD-Project/PcbRouter